Welcome to the Python Course!

This document compiles the advanced topics learned during the third week. It serves as a comprehensive study guide, detailing key concepts, syntax, and basic practical examples. All explanations and code blocks are derived directly from the weekly log repository.

Table of Contents / Topics Covered:

1. File Handling

On Day 12, we covered the fundamentals of **File Handling** in Python, including the context manager (`with` keyword), file pointers, avoiding dangling pointers, different file modes, and operations like reading, writing, and appending. We also explored copying files and implemented a practical logging script in **file_activity.py**.

1. Introduction to File Handling & The `with` Statement

File handling allows Python programs to read from and write to files on the disk, enabling data persistence.

The `with` Keyword (Context Manager)

The recommended way to open files in Python is using the `with` statement. It acts as a context manager that automatically handles resource allocation and ensures the file is properly closed after the block of code finishes executing, even if an exception or error occurs.

Without `with` (Manual Closing)

If we don't use `with`, we must manually call `.close()`. Failing to close a file can lead to resource leaks and locked files.

```
file = open("file.txt", "r")
data = file.read()
print(data)
file.close() # Easily forgotten!
```

With `with` (Auto-Closing)

The file is closed automatically as soon as the execution leaves the `with` block.

```
with open("file.txt", "r") as fp:
    data = fp.read()
    print(data)
# File is closed automatically here!
```

---

2. File Pointer & Dangling Pointer

What is a File Pointer?

A **file pointer** (e.g., `fp` in `with open(...) as fp`) is a reference or cursor that points to a specific position within the file. When a file is opened, the pointer starts at a default position (usually the beginning `0`, or the end in append mode). Any read or write operations move the file pointer forward by the number of bytes read/written.

■ Dangling File Pointer

A **dangling file pointer** or dangling reference occurs when a file pointer variable is accessed after the underlying file resource has been closed. Trying to perform I/O operations on a closed file pointer will raise a `ValueError`.

```
with open("file.txt", "r") as fp:
    data = fp.read()
# The file is now closed. fp is a dangling pointer/reference.
# Attempting to read from it now will raise an error:
try:
    fp.read()
except ValueError as e:
    print(f"Error: {e}")  # Output: I/O operation on closed file.
```

■ 3. File Modes in Python

When opening a file using `open()`, we specify a **mode** to define the allowed operations:

Mode	Name	Description	Pointer Position	File Creation	Behavior if File Exists
`'r'`	Read	Opens file for reading (default).	Beginning (`0`)	Raises `FileNotFoundError`	Opens normally
`'w'`	Write	Opens file for writing.	Beginning (`0`)	Creates new file	Truncates (clears) existing content
`'a'`	Append	Opens file for appending.	End of file	Creates new file	Keeps existing content and appends to end

---

■ 4. Reading, Writing, & Appending Operations

Reading a File (`'r'`)

Reads content from an existing file.

```
# [day_12.py](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_12.py)
with open("file.txt", "r") as fp:
    data = fp.read()
    print(data)
```

■ Writing to a File (`'w'`)

Writes new content to a file, overwriting any previous content.

```
# [day_12.py](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_12.py)
with open("file.txt", "w") as fp:
    data = "XYZ"
    fp.write(data)
```

Appending to a File (`'a'`)

Adds content to the end of a file without deleting the existing content.

```
# [day_12.py](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_12.py)
with open("file.txt", "a") as fp:
    new_data = "\nXYZ"
    fp.write(new_data)
```

5. Copying Two Files

To copy the contents of one file to another, we can combine read and write operations. We open the source file in read mode (`'r'`), read its contents, and then write those contents to the destination file in write mode (`'w'`).

```
# Copy contents from f1.txt to f2.txt
with open("f1.txt", "r") as source:
    content = source.read()
with open("f2.txt", "w") as destination:
    destination.write(content)
```

6. Practical Logging Example: Date-Based Logs

In **file_activity.py**, we implemented a script that collects user information (names and flat numbers) and logs them sequentially under a specific date in **log.txt**. It leverages the append mode (`'a'`) to ensure that new logs are added to the end of the file without overwriting existing entries.

Python Code (file_activity.py)

```
with open("log.txt","a") as register:
  date = "07/08/2026"
  heading = f"\nLogged on {date}"
  register.write(heading)
  people = 4
  # Iterate and collect details for 4 people
  for i in range(people):
    name = input("Enter the name: ")
    flat_no = int(input("Enter flat no: "))
    data = f"\n{i+1}){name} - {flat_no}"
    register.write(data)
  footer = f"\nLog completed for {date}"
  register.write(footer)
```

2. Introduction to Pandas & Data Analysis

On Day 13, we covered the basics of the **Pandas** library in Python, including how to install and import it, create DataFrames, load external CSV datasets, slice data using `iloc`, and perform basic data inspection and cleaning (detecting missing values and duplicates).

1. Installation & Setup

To use Pandas, it first needs to be installed via `pip` (Python package installer) and then imported into the script or notebook.

Installation

Run the following command in your terminal:

```
pip install pandas
```

Importing Pandas

It is a standard convention to import Pandas under the alias `pd`:

```
import pandas as pd
```

■ 2. Creating a DataFrame from a Dictionary

A **DataFrame** is a two-dimensional, size-mutable, and tabular data structure with labeled axes (rows and columns). We can create a DataFrame from a standard Python dictionary.

```
# [day_13.ipynb](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_13.ipynb)
emp_dict = {
  &#x27;e_id': [1, 2, 3, 4],
  &#x27;e_name': ["A", "B", "C", "D"],
  &#x27;e_salary': [10, 20, 15, 30]
}
emp = pd.DataFrame(emp_dict)
emp
```

Output:

```
   e_id e_name  e_salary
0     1      A        10
1     2      B        20
2     3      C        15
3     4      D        30
```

---

3. Data Slicing with `iloc`

The `.iloc` indexer is used for integer-location based indexing/selection by position.

```
# [day_13.ipynb](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_13.ipynb)
# Selecting all rows and all columns
emp.iloc[:, :]
# Selecting all rows and only the first column
emp.iloc[:, :1]
```

4. Reading CSV Files

Pandas makes it easy to read data from external files, such as CSV files, using `pd.read_csv()`.

```
# Load dataset from data.csv
dataset = pd.read_csv(&#x27;data.csv')
```

5. Data Inspection & Summarization

Once a dataset is loaded, we can use several built-in Pandas methods to inspect and summarize the data.

`head()` and `tail()`

- `head(n)` returns the first `n` rows of the DataFrame (defaults to 5).
- `tail(n)` returns the last `n` rows of the DataFrame (defaults to 5).

```
# View the first 5 rows
dataset.head()
# View the last 5 rows
dataset.tail()
```

`describe()`

The `describe()` method generates descriptive statistics of the numerical columns in the DataFrame, including count, mean, standard deviation, min, max, and percentiles.

```
# Generate summary statistics
dataset.describe()
```

6. Basic Data Cleaning

Checking for Missing Values (`isna().sum()`)

The `isna()` method detects missing values, and appending `.sum()` counts the number of missing (NaN) values per column.

```
# Count missing values in each column
dataset.isna().sum()
```

Checking for Duplicate Rows (`duplicated().sum()`)

The `duplicated()` method returns a boolean Series denoting duplicate rows, and `.sum()` counts the total number of duplicate rows.

```
# Count duplicate rows
dataset.duplicated().sum()
```

3. Data Cleaning & Feature Engineering

On Day 14, we covered data cleaning and feature engineering using **Pandas**, including checking for missing (null) values, handling duplicates, creating new columns through feature engineering, and saving the processed data back to a CSV.

1. Handling Null (Missing) Values

Missing values are identified using the `isna()` method. We can check the count of null values per column and fill them using `fillna()`.

Detecting Null Values

To count the number of missing values in each column of the **orders.csv** dataset:

```
# [day_14.ipynb](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_14.ipynb)
data.isna().sum()
```

■ Filling Missing Values (`fillna()`)

We filled missing values in the dataset using different techniques:

- **Quantity**: Replaced missing values with a default value of `1`.
- **Price**: Replaced missing values with the mean of the `Price` column.
- **Discount**: Replaced missing values with a default value of `50`.

```
# [day_14.ipynb](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_14.ipynb)
# Fill missing Quantity with a constant
data[&#x27;Quantity'] = data['Quantity'].fillna(1)
# Fill missing Price with the column mean
data[&#x27;Price'] = data['Price'].fillna(data['Price'].mean())
# Fill missing Discount with a constant
data[&#x27;Discount'] = data['Discount'].fillna(50)
```

---

2. Handling Duplicate Values

Duplicate entries can distort data analysis. We check for and drop duplicate rows to ensure data quality.

Counting Duplicate Rows

To find the number of duplicate rows in the dataset:

```
# [day_14.ipynb](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_14.ipynb)
data.duplicated().sum()
```

■ Dropping Duplicate Rows

To remove the duplicates:

```
# [day_14.ipynb](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_14.ipynb)
data.drop_duplicates()
```

■ 3. Feature Engineering

Feature engineering is the process of creating new features from existing data. We created two new fields:

1. **`Total Price_BD`**: Total price before discount (**`Quantity * Price`**).
2. **`Total Price`**: Total price after subtracting the discount (**`Total Price_BD - Discount`**).

```
# [day_14.ipynb](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_14.ipynb)
# Calculate total price before discount
data[&#x27;Total Price_BD'] = data['Quantity'] * data['Price']
# Calculate total price after discount
data[&#x27;Total Price'] = data['Total Price_BD'] - data['Discount']
```

4. Saving Processed Data

Once data cleaning and feature engineering are complete, the resulting DataFrame is saved to a new CSV file **processed_orders.csv**:

```
# [day_14.ipynb](file:///c:/Users/vijay/Desktop/pythonCourse/week_03/day_14.ipynb)
data.to_csv(&#x27;processed_orders.csv')
```

4. Introduction to DBMS & PostgreSQL Basics

On Day 15, we covered the basics of Database Management Systems (DBMS), distinguishing between structured and unstructured data, and learned how to interact with a relational database using PostgreSQL, focusing on creating tables, inserting data, and selecting data.

■ 1. Structured vs. Unstructured Data

Data in the modern digital world can be categorized primarily into structured and unstructured formats:

- **Structured Data**: Data that conforms to a strict, pre-defined schema or model. It is highly organized and typically stored in relational database tables with rows and columns. Examples include SQL databases, CSV files, and Excel spreadsheets.
- **Unstructured Data**: Data that does not have a predefined format or organizational structure. It cannot be easily fit into a relational database table. Examples include text files, emails, images, videos, audio recordings, and PDFs.

2. PostgreSQL Basics

PostgreSQL (often simply Postgres) is a powerful, open-source object-relational database management system (ORDBMS). It is known for its reliability, feature robustness, and performance.

■ Creating a Table (`CREATE TABLE`)

To store structured data, we define a table schema with columns and their corresponding data types.

```
-- Example: Creating a students table
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT,
    grade VARCHAR(2),
    enrolled_date DATE DEFAULT CURRENT_DATE
);
```

Inserting Data (`INSERT INTO`)

Once a table is created, we populate it by inserting rows of data.

```
-- Example: Inserting a single record
INSERT INTO students (name, age, grade)
VALUES (&#x27;Vijay', 21, 'A');
-- Example: Inserting multiple records
INSERT INTO students (name, age, grade)
VALUES
```

```
    (&#x27;Arjun', 22, 'B'),
    (&#x27;Priya', 20, 'A');
```

Selecting Data (`SELECT`)

We retrieve data from the database using queries.

```sql
-- Retrieve all columns and rows from the table
SELECT * FROM students;
-- Retrieve specific columns for students older than 20
SELECT name, grade
FROM students
WHERE age > 20;
```

5. SQLite3 & Authentication Simulation

On Day 16, we covered using the built-in **`sqlite3`** module in Python to interact with relational databases. We focused on connection establishment, database cursors, query execution, transactions, and built a command-line application in **main.py** to simulate user registration, login, and administration.

1. Connection Establishment & Cursor

SQLite is a serverless, self-contained database engine. Python has native support for SQLite via the **`sqlite3`** library.

- **Connecting to Database**: We use **`sqlite3.connect('database_name.db')`** to establish a connection. If the database file does not exist, SQLite will automatically create it.
- **Database Cursor**: A cursor object is created from the connection object via **`conn.cursor()`**. It serves as the gateway to execute SQL commands and retrieve query results.

```
import sqlite3
# Establish connection to the database
conn = sqlite3.connect(&#x27;facebook.db')
# Create a cursor object
cursor = conn.cursor()
```

■ 2. Executing Queries & Transaction Management

- **Executing Queries**: The **`execute()`** method on the cursor runs SQL commands. When inserting values, we pass parameters as a tuple to protect against SQL Injection.
- **Committing Changes**: For write operations (**`INSERT`**, **`UPDATE`**, **`DELETE`**), changes must be committed using **`conn.commit()`** to save them permanently to the database.
- **Fetching Records**:
  - **`cursor.fetchone()`**: Retrieves the next single row from the result set, or **`None`** if no more rows are available.
  - **`cursor.fetchall()`**: Retrieves all matching rows from the query results as a list of tuples.

Basic Example (day_16.py)

```
import sqlite3
# Connect to database and retrieve a cursor
conn = sqlite3.connect(&#x27;test.db')
cursor = conn.cursor()
# Insert data
query = "INSERT INTO users(id, name) VALUES(104,&#x27;XYZ'),(102,'Ajay'),(103,'ABC');"
cursor.execute(query)
conn.commit()
# Query and fetch data
query = "SELECT * FROM users;"
cursor.execute(query)
data = cursor.fetchall()
for row in data:
  print(*row)
```

3. Authentication Simulation Program

In main.py, we built a command-line interface mimicking user registration, user login, and administrative listing of active records on a SQLite database.

Feature Breakdown:

1. `createTable()`: Sets up the table structure if the table doesn't already exist.
2. `register()`: Prompts for credentials, inserts them with default follows and following counts set to `0`, and commits the transaction.
3. `login()`: Queries the record corresponding to the username, verifies the existence of the user, and validates both mail and password credentials.
4. `adminDisplay()`: Retrieves all records from the database and prints them out.

Python Code (main.py)

```
import sqlite3
conn = sqlite3.connect(&#x27;facebook.db')
cursor = conn.cursor()
def createTable():
  query = &#x27;CREATE TABLE IF NOT EXISTS users(username VARCHAR(50), mail VARCHAR(50), password VARCHAR(15
  cursor.execute(query)
  print("Create Table - Users -")
def adminDisplay():
  query = "SELECT * FROM users;"
  cursor.execute(query)
  users = cursor.fetchall()
  for user in users:
    print(*user)
def login():
  username = input("Enter username: ")
  mail = input("Enter mail: ")
  password = input("Enter password: ")
  # Collecting details using parameter query
  query = "SELECT * FROM users WHERE username = ?;"
  cursor.execute(query, (username,))
  user_details = cursor.fetchone()
  if not user_details:
    print("No such user exists.")
    return
  correct_mail = user_details[1]
  correct_pwd = user_details[2]
  # Credential Comparison
  if mail == correct_mail:
    if password == correct_pwd:
      print("Login done. Welcome back.")
    else:
      print("Incorrect password.")
  else:
    print("Incorrect mail.")
def register():
  username = input("Enter username: ")
  mail = input("Enter mail: ")
  password = input("Enter password: ")
  query = "INSERT INTO users(username, mail, password, follows, following) VALUES(?,?,?,0,0);"
  cursor.execute(query, (username, mail, password))
  conn.commit()
  print("Registration completed. Welcome to facebook.")
choice = 0
```

```
while choice != 4:
  choice = int(input("Select choice: \n 1)Login 2)Register 3)Admin 4)Exit. Make your choice: "))
  if choice == 1:
    login()
  elif choice == 2:
    register()
  elif choice == 3:
    adminDisplay()
```

Practice Questions

Part A: File Handling & Pandas

Question 1: Word Counter

Write a function **count_words(filename)** that takes a text file's name as input and returns the total number of words in that file.

```
Example
Input:
file.txt (containing &#x27;Hello World from Python')
Output:
4
```

Question 2: Filter Orders by Price

Write a python function using Pandas that reads a CSV file **orders.csv**, fills missing **Quantity** values with a default value of 1, and filters the rows to keep only orders where the **Price** is greater than 100. Return the clean, filtered DataFrame.

```
Example
Input:
orders.csv
Output:
Filtered DataFrame with non-null quantities and prices > 100
```

Question 3: Feature Engineering & De-duplication

Write a function that accepts a Pandas DataFrame, creates a new feature column **Total_Cost** calculated as **Quantity * Price**, removes any duplicate rows from the DataFrame, and returns the modified DataFrame.

Part B: Relational Databases (PostgreSQL & SQLite)

Question 4: SQL Table Definition

Write a SQL query to create a table named **inventory**. The table must have: **product_id** (SERIAL, PRIMARY KEY), **name** (VARCHAR, NOT NULL), **price** (NUMERIC, greater than 0), and **stock** (INTEGER, default 0).

Question 5: Fetch Users from SQLite Database

Write a python function **fetch_all_users()** that connects to an SQLite database named **facebook.db**, executes a SELECT statement to retrieve all usernames and email addresses from the **users** table, and prints each user.

```
Example Output:
Ajay ajay@mail.com
Vijay vijay@mail.com
```

Simulation Question

Problem Statement

Design a Python program using SQLite3 to manage a simple Bookstore Inventory.

Write a function named **bookstore()** that acts as the main program. Inside **bookstore()**, define the following nested helper functions:

- **add_book(title, author, price)** – Inserts a new book record into a table named **books**.
- **view_books()** – Fetches and displays all books in the inventory.
- **search_book(title)** – Searches and displays details of a book based on its title.

Database Schema

The table **books** should contain the following fields: **title** (VARCHAR), **author** (VARCHAR), and **price** (FLOAT). Ensure the table is created automatically using a **CREATE TABLE IF NOT EXISTS** query at startup.

Sample Run

```
Input / Operations:
Welcome to the Bookstore Inventory System
Select Choice:
1) Add Book  2) View Books  3) Search Book  4) Exit
Make your choice: 1
Enter title: Python Basics
Enter author: Guido van Rossum
Enter price: 450.0
Book added successfully.
Make your choice: 2
--------- BOOKS IN INVENTORY ---------
Python Basics - Guido van Rossum - Rs. 450.0
--------------------------------------
```